

INTEGRATION GUIDE · MAY 2026



PrintLoop

New Frontend, Wired

Connecting the editorial frontend to your working backend.

FOR

Abdurrahman

What you just received

This package contains a complete new frontend for PrintLoop, built in the editorial-energetic v3 style you locked in. It runs out of the box, looks polished, and is wired to talk to your existing backend at `http://localhost:3000`.

Tonight you don't need to throw away your old frontend. You can run them side by side, see how the new one feels, and decide whether to replace your old one when you're ready.

IMPORTANT — This guide assumes your backend is already running. If it isn't, start it first: `cd backend`, then `pnpm dev`. See your existing setup manual for that part.

Table of contents

- ▶ Part 1 — Install the new frontend (5 min)
- ▶ Part 2 — Configure it to talk to your backend (1 min)
- ▶ Part 3 — Run it and try the full flow
- ▶ Part 4 — What's wired vs what's still mock data
- ▶ Part 5 — Replace the old frontend when you're ready
- ▶ Appendix A — File structure overview
- ▶ Appendix B — When something goes wrong
- ▶ Appendix C — Handing it to a freelancer

Part 1 — Install the new frontend

Step 1.1 — Extract the package

1. Find `printloop-new-frontend.zip` in your Downloads folder.
2. Right-click → Extract All → pick a location. Suggestion:

```
C:\Users\abdur\Downloads\PrintLoop-FINAL\PrintLoop-FINAL\frontend-new\
```

Notice we are putting it in a NEW folder called `frontend-new` — not replacing the existing frontend. We keep both.

Step 1.2 — Open it in VS Code

1. In VS Code: File → New Window
2. File → Open Folder → navigate to the `frontend-new` folder → Open

3. You should see folders: src, public, package.json, etc.

Step 1.3 — Install dependencies

Open the terminal in VS Code (Ctrl+`) and run:

```
pnpm install
```

Or if you prefer npm:

```
npm install
```

Either works. Takes 2-3 minutes. ~250MB download. Use WiFi if you can.

Part 2 — Configure the API URL

The frontend needs to know where your backend lives. There's a file called `.env.development` in the root.

Step 2.1 — Open `.env.development`

In the VS Code file tree, click `.env.development`. You should see exactly one line:

```
VITE_API_URL=http://localhost:3000/api
```

Step 2.2 — Check that matches your backend

This should already be correct because we know your backend runs on port 3000 with the `/api` prefix. If your backend uses a different port, change the number here.

IMPORTANT — Don't add a trailing slash. `http://localhost:3000/api` — not `http://localhost:3000/api/`.

Part 3 — Run it

Step 3.1 — Make sure your backend is running

In one PowerShell window:

```
cd C:\Users\abdur\Downloads\PrintLoop-FINAL\PrintLoop-FINAL\backend pnpm dev
```

Wait for: 🚀 Server is running on <http://localhost:3000>

Step 3.2 — Start the new frontend

In a SECOND PowerShell window (or in VS Code's terminal):

```
cd C:\Users\abdur\Downloads\PrintLoop-FINAL\PrintLoop-FINAL\frontend-new pnpm dev
```

You should see:

```
VITE v5.x.x ready in 300 ms → Local: http://localhost:5173/
```

IMPORTANT — If your OLD frontend was running on 5173, stop it first (Ctrl+C in its terminal) — only one can use the port at a time.

Step 3.3 — Open the new app

Browser → <http://localhost:5173>

You should see the new editorial landing page — cream paper, big serif headline, persimmon CTA, scrolling marquee at top.

Step 3.4 — Try the full flow

1. Click REGISTER on the landing page
 2. Fill in the form. Use a NEW email — the backend rejects duplicates.
 3. Submit. You should get redirected to the verify-email page with your email pre-filled.
 4. Check your Gmail inbox for the 6-digit code.
 5. Paste it into the 6 boxes (you can paste all 6 at once into the first box).
 6. Click VERIFY EMAIL. You should get redirected to login.
 7. Sign in with the email + password you just used.
 8. Welcome to the dashboard.
-

Part 4 — What's wired vs what's mock data

✓ Fully wired and working

- ▶ User registration → POST /api/customer/auth/register
- ▶ Email verification → POST /api/customer/auth/verify-email
- ▶ Resend verification → POST /api/customer/auth/send-verification-email
- ▶ Sign in → POST /api/customer/auth/login
- ▶ Forgot password → POST /api/customer/auth/forgot-password
- ▶ Auto token refresh → POST /api/customer/auth/refresh
- ▶ Sign out (frontend only — clears local tokens)
- ▶ Protected routes (you can't see dashboard without signing in)

UI built, using mock data

These screens look real and work visually, but show demo data because the backend endpoints either don't exist yet or need different field shapes. They're easy to wire up once you decide the data shapes.

- ▶ Dashboard active job + stats (uses hardcoded demo values)
- ▶ Print jobs list (uses hardcoded sample jobs)
- ▶ Wallet balance + transactions (uses hardcoded amounts)
- ▶ Stations directory (uses hardcoded station list)
- ▶ New print flow (does NOT actually upload or charge — the receipt screen is a UI mock)

✗ Not implemented yet

- ▶ Actual file upload to Cloudinary
- ▶ Actual Paystack payment integration
- ▶ Kiosk-side code verification (the kiosk page is UI-only)
- ▶ Group printing flow
- ▶ Admin panel

WHY THIS MATTERS — The auth flow being fully wired is the important part. Everything else is connecting the existing UI to backend endpoints that may not exist yet — that's a 2-3 day project for a freelancer following the structure already in place.

Part 5 — Replace the old frontend when you're ready

Right now you have two frontends: the OLD one (your real working print-loop-customers-main) and the NEW one (this editorial version). The OLD one has more wiring; the NEW one has nicer design but less integration.

Strategy A — Run both, learn, then commit

Best path. For a few days:

- ▶ Use the OLD frontend when you want to actually upload, pay, or print something.
- ▶ Use the NEW frontend to show people the design, get feedback, iterate.
- ▶ After 1-2 weeks: decide which to keep.

Strategy B — Move forward with the NEW frontend

If you're confident, archive the old frontend and proceed:

1. Rename the old frontend folder: frontend → frontend-old
2. Rename the new one: frontend-new → frontend
3. Hire a freelancer for ₦150-250k to wire up the remaining endpoints (print jobs, wallet, file upload, payment).
4. Send them this guide + the freelancer brief in Appendix C.

Strategy C — Apply the NEW look to the OLD frontend

Lower risk: keep all your existing wiring, just restyle. The design tokens, components, and CSS classes from the new frontend can be copied into the old one. This is the ₦100-150k option.

Appendix A — File structure

Here's what's in the new frontend, in plain English:

File / folder	What it does
package.json	List of libraries the project needs
vite.config.ts	Build tool config

tailwind.config.js	Design system — colors, fonts, animations
src/index.css	Component styles (buttons, inputs, cards)
src/main.tsx	Entry point — boots the app
src/App.tsx	Main router — which URL shows which page
src/components/ui/	Reusable UI pieces (Button, Input)
src/components/layout/	Page frames (Masthead, AppLayout, AuthLayout)
src/pages/auth/	Login, Register, Verify, Forgot
src/pages/customer/	Dashboard, NewPrint, Jobs, Wallet, etc.
src/pages/kiosk/	Kiosk tablet screen
src/store/	Redux state (auth, API calls)
src/store/services/	RTK Query — the part that talks to the backend
src/constants/	Route URLs, config values
src/lib/errors.ts	Helper to display backend errors nicely

Appendix B — When something goes wrong

"Cannot connect to backend" or empty data

- ▶ Backend isn't running. Start it: `cd backend && pnpm dev`
- ▶ Wrong API URL. Check `.env.development` matches your backend port.
- ▶ CORS error in browser console. Add `http://localhost:5173` to your backend's CORS allowed origins.

"Port 5173 is in use"

Your old frontend is still running. Stop it (Ctrl+C in its terminal).

"npm install fails"

- ▶ Bad network. Switch to WiFi. Re-run.
- ▶ Different node version needed. The project works on Node 18+ and pnpm 8+.

"Registration fails with: Route not found"

- ▶ Your VITE_API_URL is wrong. Should be: `http://localhost:3000/api`
- ▶ Backend isn't actually running on 3000 — check the backend terminal output.

"Login fails with: No token provided"

- ▶ This is the bug we already solved — the backend expects `/api/customer/auth/login`. This frontend is configured for that. If you still see it, check that `.env.development` is loaded (restart pnpm dev after changes to `.env` files).

"Verification fails: Invalid token"

- ▶ Code expired (10 min limit). Click RESEND CODE.
- ▶ Wrong code copied. Check carefully.

Appendix C — Handing it to a freelancer

To wire the remaining features (file upload, payment, kiosk-side code verification, admin panel), here's a brief you can send to any Nigerian Node.js + React freelancer:

BRIEF — I have a working backend (Node + Express + TypeORM + MySQL) at `http://localhost:3000` with all customer auth endpoints wired. I have a new React + Vite + Tailwind + RTK Query frontend with all the UI built. I need you to: (1) wire the file upload flow to `POST /api/customer/files/upload`, (2) wire the payment flow to Paystack via the backend's existing payment endpoints, (3) wire the print jobs list to `GET /api/customer/print-jobs`, (4) wire the wallet to the existing wallet endpoints, (5) build a kiosk-side code verification page that calls the backend's printer endpoints. UI is already built — you only need to connect data. Estimate: 15-25 hours. Budget: ₦200-300k.

Send them: the zipped new frontend folder, this guide, and the backend folder zipped (so they can run both and test).

Where to find them

- ▶ Upwork — <https://upwork.com> (search Nigerian Node.js + React)
- ▶ X (Twitter) — search Lagos React developer or Nigerian fullstack
- ▶ Genesys / Tunga — Nigerian dev agencies



End of guide.

Stop queueing. Start printing.